

Capstone Project Guidelines & Rubric

Web Technologies SP26 · BSCS 4th Semester · Sections 1M & 2M

Individual Capstone Project | Total: 50 + 5 bonus marks

Overview

This is your individual capstone project for the Web Technologies course. Each student builds a complete web application of their chosen domain, using JavaScript with JSON Server as a mock REST backend. The project must demonstrate concepts covered this semester i.e., DOM manipulation, events, forms, validation, fetch with async/await, and full CRUD operations.

1. Technology Stack — Mandatory for Everyone

All projects must use the following stack. No exceptions, regardless of what you may have proposed earlier.

Layer	Required
Markup	HTML5 - semantic, accessible structure
Styling	Custom CSS - OR Bootstrap 5 (Bootstrap earns +5 bonus marks)
JavaScript	Plain JavaScript ONLY - no React, no Vue, no jQuery, no frameworks
Backend	JSON Server - local mock REST API (mandatory)
Data format	JSON - accessed via fetch + async/await

Important

Students who initially proposed React must switch to plain JS. The course objective is to demonstrate JavaScript fundamentals and not framework usage. Frameworks will be covered in later courses.

2. Project Architecture

Every project, regardless of domain, must follow this structure:

File	Purpose
index.html	User-facing page (main interface)
admin.html	Admin/management page (CRUD controls, stats)
style.css	All styling (or Bootstrap-based)
app.js	User panel JavaScript
admin.js	Admin panel JavaScript

File	Purpose
db.json	JSON Server data file
README.md	Setup, run instructions, feature list, screenshots

Your data must be stored in `db.json` and accessed only through JSON Server. **No hardcoded arrays** inside your JS files for the main data.

3. Required Features — 45 Marks

These features are mandatory. Missing any of them costs heavy marks. Adapt them to your specific project domain.

3.1 User Panel (`index.html`)

- Fetch and display a list of resources using `GET` from JSON Server
- Filter the list by at least **one** attribute (category, status, type, date, etc.)
- Submit a new resource via a form using `POST` — form must have at least **5 input fields**
- Inline form validation — required fields, format checks (no `alert()` boxes)
- List re-renders automatically after successful submission
- Show loading state while fetching, error state if JSON Server is unreachable

3.2 Admin Panel (`admin.html`)

- Fetch and display **all** resources (including any hidden from users)
- Edit a resource — load it into a form, save changes using `PUT` or `PATCH`
- Delete a resource using `DELETE` — with a confirmation dialog
- Display at least **3 summary statistics** (counts, percentages, averages, totals)
- Visual distinction from user panel (different navigation colour, clear admin label)

3.3 Code Requirements (across both panels)

- All four HTTP methods used appropriately: `GET`, `POST`, `PUT` (or `PATCH`), `DELETE`
- Use of `async/await` with `try/catch` — not raw `.then()` chains
- Check `response.ok` before parsing JSON
- Clean, semantic HTML — appropriate use of `<nav>`, `<main>`, `<section>`, `<form>`, labels for inputs
- Working `README.md` with setup steps and how to run

4. Recommended Features — Bonus +5 Marks

Students who finish all required features can attempt one of the following for bonus marks. Only the best attempt counts (no stacking).

- **Bootstrap styling** used consistently across both panels (not mixed with custom CSS) — **+5 marks**
- Search bar with **debounced** input
- Sort by multiple criteria (with sort direction toggle)
- Pagination for long lists
- Export filtered data to CSV or JSON
- Dark mode toggle, persisted in `localStorage`
- Edit confirmation modal instead of inline edit mode
- Empty state illustrations or icons when no data exists
- Mobile-responsive layout that works well on phones

Bonus rule

Bonus is awarded for ONE clean implementation, not many half-done attempts. Pick the feature that best fits your project and execute it well.

5. Marking Rubric — 50 + 5 Marks

Criterion	What earns full marks	Marks	Sub-breakdown
Functionality & Features	All required features work end-to-end without crashes. Form validation prevents bad data. Edit and delete flows complete and tested.	15	User CRUD: 5 Admin CRUD: 5 Validation: 3 Stats: 2
UI/UX + Styling	Clean visual hierarchy, consistent spacing, readable typography, responsive on common screen sizes, no broken layouts. Loading and error states are clearly visible.	10	Layout & spacing: 4 Typography: 2 Responsiveness: 2 Load/error states: 2
Code Quality & Structure	Meaningful variable names, helper functions extracted (e.g. <code>renderList</code> , <code>showError</code>), no copy-pasted blocks, no dead code or <code>console.logs</code> left in.	5	Naming/readability: 2 Function extraction: 2 No dead code: 1
API Integration (JSON Server)	All four HTTP methods used correctly. Query parameters where appropriate. Proper Content-Type headers on POST/PUT. <code>response.ok</code> checked. <code>try/catch</code> around every fetch.	10	GET (list+filter): 3 POST: 2 PUT/PATCH: 2 DELETE: 1 Error handling: 2
Demo + Viva Explanation	Student walks through their own code, explains trade-offs, answers questions about <code>fetch</code> , REST methods, status codes, <code>async/await</code> , event loop. Demonstrates the app live without coaching.	10	Live demo: 4 Code explanation: 3 Concept questions: 3
Bonus (Bootstrap OR a Recommended Feature)	Bootstrap used throughout consistently, OR one recommended feature implemented cleanly and integrated. See Section 4.	+5	All-or-nothing
Total		50+5	

6. Viva Format — Individual, ~10 Minutes

Each student gets a private 10-minute viva. The viva is the single most important component of your grade — even a working project earns low marks if the student cannot explain their own code.

Viva Structure

Phase	Time	What happens
1. Live Demo	3 min	Run JSON Server, open the app, demo user panel → admin panel → CRUD operations. App must work on first try.
2. Code Walkthrough	3 min	Show 2–3 functions you wrote. Explain what they do, line by line. Instructor picks the functions, not you.
3. Concept Questions	3 min	Questions on: fetch, async/await, REST methods, status codes, event loop, JSON, validation, DOM.
4. Reflection	1 min	What was the hardest part? What would you improve given more time?

Sample Concept Questions

- What does `e.preventDefault()` do, and why do we call it inside a form submit handler?
- Why does `setTimeout(cb, 0)` not run immediately? Walk me through the event loop.
- What is the difference between `PUT` and `PATCH`?
- What does `JSON.stringify()` return, and why do we need it before sending a POST?
- If your fetch returns a 404, will it throw an error? How do you handle it?
- Show me your validation code. Why didn't you use the browser's built-in validation?
- Why is `response.json()` called with parentheses? What does it return?

Honest warning

If you cannot explain a portion of your code, you will be marked as if that portion does not exist — even if it works. This rule is to discourage copy-pasted or AI-generated code that the student does not understand.

Tip

Submit early if you finish early. You can resubmit improvements until the deadline, but late submissions are not accepted.

7. Submission Checklist

Submit a single zip file named `WebProject_<RollNo>.zip`. Before submitting, verify ALL of the following:

- ✓ Project folder contains: `index.html`, `admin.html`, `style.css`, `app.js`, `admin.js`, `db.json`, `README.md`
- ✓ README has: project description, how to install + run, list of features, screenshots
- ✓ Running `npx json-server --watch db.json` works from your project folder
- ✓ Opening `index.html` in a browser shows data fetched from JSON Server

- ✓ Admin panel CRUD operations work — tested at least once for each method
- ✓ **NO** `node_modules` folder in the zip
- ✓ **NO** hardcoded arrays for main data inside JS files
- ✓ No `console.log` statements left in your code (except intentional ones)
- ✓ Your name and roll number are in the README header

8. Deductions & Academic Honesty

Issue	Penalty
App crashes during viva	−3 marks
JSON Server not running / not configured properly	−5 marks
Hardcoded data instead of fetched from JSON Server	−5 marks
README missing or unusable	−2 marks
Late submission	Not accepted
Code the student cannot explain in viva	Treated as missing
Plagiarism (copied code from another student or full AI-generated submission)	Zero on project + referred to academic committee

Use of AI tools — clear policy

You may use AI tools (ChatGPT, Claude, Copilot) to LEARN concepts, debug errors, and understand code. You may NOT submit AI-generated code that you cannot fully explain. The viva determines whether your understanding matches your submission. If you cannot walk through any part of your code, that part scores zero.

9. Frequently Asked Questions

Q: Can I use Bootstrap and still customise the colours?

Yes — you can use Bootstrap's utility classes and override specific colours via CSS variables or a small custom stylesheet. The bonus is forfeited only if you bypass Bootstrap entirely for layout.

Q: My project is a portfolio / single-page website. Do I still need two HTML files?

Yes. A portfolio is still required to have a user-facing page (visitor view) and an admin page where you manage your own content (add/edit/delete projects, skills, experience). The admin panel demonstrates your CRUD skills.

Q: My project is a game or quiz. How does CRUD fit?

. For a quiz app: admin creates/edits/deletes questions; user takes the quiz. For a game, no admin panel given time duration.

Q: What if JSON Server fails during my viva?

Test it 24 hours before. Have JSON Server installed and the command ready. If it fails during the viva because of your setup, you lose 5 marks. If it fails due to a system issue beyond your control, the viva is rescheduled.

Q: Can I use icons or external libraries (Chart.js, etc.)?

Yes — icons (Font Awesome, Bootstrap Icons), Chart.js for dashboards, date pickers — all welcome via CDN. The only restriction is on JS frameworks (no React, Vue, Angular, jQuery).

Q: My project domain is e-commerce, same as 4 others. Will my marks suffer?

No. Marks are based on your implementation, not the topic. Same-topic projects are marked independently — what differs is execution, UX choices, and feature depth.